COMPANION READER

Computer Vision Fundamentals & Applications

From pixels to edges — images, filters, and features, explained simply

Chapter 5 · Read alongside the lecture slides · Every slide example worked out in full

Contents

1 The big idea: image processing vs. computer vision 1

2 Pixels, channels, and shape: what a computer sees 3

3 Three flavors of image: color, grayscale, binary 4

4 Reading and reshaping images: libraries, resize, translate, rotate 5

5 Cleaning up noise: Gaussian and median filtering 6

6 Drawing the line: simple and adaptive thresholding 8

7 Finding edges: convolution and kernels 9

8 A cleaner edge: the Canny recipe 11

9 Reshaping shapes: erosion and dilation 12

Quick self-test 14

Glossary & symbols 14

Further reading 15

How to use this companion. This reader walks beside the lecture slides and fills in what a fast-moving deck cannot: the plain-English why, and every calculation worked out with real numbers. Five colored boxes flag five different jobs. Blue is **the intuition** — the idea in plain words. Amber is **everyday picture** — a real-life analogy for the same idea. Green is a **worked example** — real numbers, every step shown. Red is **watch out** — the mistake beginners make here. Purple is the **key takeaway** — the one line to remember. Read them in that order and the math never feels sudden.

1 The big idea: image processing vs. computer vision

A camera gives a computer a huge grid of numbers. What the computer does with that grid splits into two jobs, and this whole course lives inside that split. **Image processing** changes the picture: it blurs it, sharpens it, resizes it. **Computer vision** goes one step further: it decides what the picture *means* — "that is a dog", "that sign says STOP".

☞ The intuition

Image processing changes a picture. Computer vision reads a picture. One works on pixels; the other works on meaning.

★ Everyday picture

Think of a photo editor and a friend looking over your shoulder. The editor (like Photoshop) crops, brightens, and sharpens — it never tells you what is in the photo. Your friend looks at the same photo and says, "that's your dog at the beach." The editor is image processing. Your friend is computer vision.

Every computer-vision system still needs image processing first: you almost always clean up, resize, and filter an image *before* asking a model what it shows. So the two jobs are not rivals; processing is step one, vision is step two.

✎ Worked example: sorting slide examples by job

The slides showed four example outputs and asked: is this image processing, or computer vision? Here is how to tell them apart, one at a time.

Step 1. **Look at a rotated photo (a shear transform).** The pixels moved, but nothing was labeled or recognized. **Verdict: image processing.**

Step 2. **Look at a system comparing "computer vision" and "machine vision."** This is about what the system understands, not about changing pixels. **Verdict: computer vision.**

Step 3. **Look at a system explaining what computer vision means, with examples.** Again, this is about meaning and understanding. **Verdict: computer vision.**

Step 4. **Look at a thresholded, segmented image.** The pixels were converted from gray to black-and-white — no object was named. **Verdict: image processing.**

The one-line test: did the output just *look different*, or did it come with a *label, a name, or a decision*? Different look ⇒ processing. A label ⇒ vision.

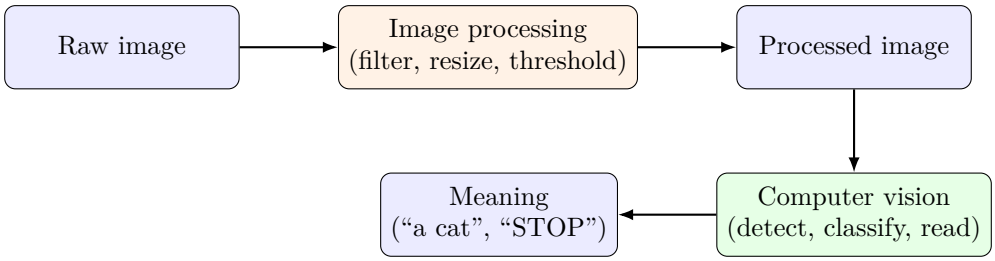


Figure 1: How to read it: image processing only reshapes the picture (top row); computer vision goes one step further and turns the picture into a meaning (bottom row). Most real systems do both, in this order.

Where this shows up in ML. A convolutional neural network's early layers behave a lot like hand-built image-processing filters — they blur, sharpen, and detect edges. Its later layers do the computer-vision job: turning those processed features into a class label. Modern deep learning quietly fuses both jobs into one trained pipeline.

✗ Watch out

Don't assume any line of OpenCV code counts as "computer vision." Resizing, blurring, and thresholding are image processing, even when you did them with an AI library.

✔ Key takeaway

Image processing reshapes pixels. Computer vision assigns meaning to them. Real systems do processing first, vision second.

2 Pixels, channels, and shape: what a computer sees

A computer never sees a "picture." It sees a big table of numbers. Each cell in that table is one **pixel**, short for "picture element," and its value tells the computer how bright that one tiny spot is.

☞ The intuition

An image is a spreadsheet of brightness numbers. Every number sits between 0 (fully dark) and 255 (fully bright). A color image needs three such spreadsheets stacked together, one each for red, green, and blue.

★ Everyday picture

Picture a mosaic made of tiny colored tiles. Stand close and each tile is just one flat color. Step back and the tiles blend into a picture. A pixel is one tile; the image is the whole mosaic.

For a grayscale image, the shape is *height* × *width*: one number per pixel. For a color image, we add a third number, the *channel* count C, almost always $C = 3$ (one channel each for red, green, blue). So a color image's shape is

$$H \times W \times C,$$

where H is the height in pixels, W is the width in pixels, and C is the number of color channels. Note the order: height comes *before* width — this trips almost everyone up the first time.

✎ Worked example: finding an image's array shape

The slide's quiz asks: an image is 700 pixels wide, 550 pixels tall, with 3 channels (red, green, blue). What is its NumPy array shape?

Step 1. **Write down what we know.** Width $W = 700$. Height $H = 550$. Channels $C = 3$.

Step 2. **Recall the order.** A NumPy image array is always ordered *height, width, channels* — rows first, then columns, then color.

Step 3. **Plug in the numbers.** Shape $= (H, W, C) = (550, 700, 3)$.

Step 4. **Sense-check.** 550 rows of pixels, each row 700 pixels wide, each pixel holding 3 numbers. That is a lot of pixels stacked in rows — exactly what a tall, wide photo should look like.

So the shape is **(550, 700, 3)** — height first, not width first.

✎ Worked example: counting total pixels

A second slide quiz: an image is 393 pixels wide and 312 pixels tall. How many pixels does it hold in total?

Step 1. **Write down what we know.** Width = 393. Height = 312.

Step 2. **State the rule.** Total pixels = width × height, since every row has the same number of pixels and there are that many rows.

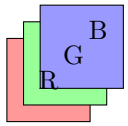
Step 3. **Multiply.** $393 \times 312 = 122{,}616$.

Step 4. **Sense-check.** $393 \times 300 \approx 118{,}000$, close to our answer, so the arithmetic looks right.

The image holds **122,616 pixels** in total (before we even count the 3 color values per pixel).

34	201	88	5
120	76	250	63
9	180	44	210

A grayscale patch: one number per pixel



A color image: 3 stacked grids

Figure 2: How to read it: left, a grayscale image really is a grid of brightness numbers. Right, a color image stacks three such grids — red, green, blue — so its shape carries a third dimension, the channel count.

Where this shows up in ML. A convolutional neural network's very first input is exactly this array. Getting the shape wrong — feeding (W, H, C) where the model expects (H, W, C), or the reverse — is one of the most common bugs when wiring up a new dataset.

✗ Watch out

OpenCV loads color images in **BGR** order (blue, green, red), not the RGB order most other libraries expect. Display a freshly loaded OpenCV image in a library that assumes RGB, and the reds and blues swap — a face can turn a strange shade of blue.

✔ Key takeaway

An image's shape is height × width × channels. Height always comes first, and OpenCV's channel order is BGR, not RGB.

3 Three flavors of image: color, grayscale, binary

Not every image needs 3 numbers per pixel. Some tasks only need brightness; some only need "is this pixel part of the shape or not." That is the difference between color, grayscale, and binary images.

☞ The intuition

Color needs 3 numbers per pixel (red, green, blue). Grayscale needs 1 number per pixel (brightness only). Binary needs the least information of all: just 0 or 255, on or off.

★ Everyday picture

A color photo is a full painting. A grayscale photo is a pencil sketch — shading, but no color. A binary image is a stencil or a silhouette — just the outline, filled solid, no shading at all.

A grayscale pixel's value is one number, from 0 (black) to 255 (white). To go from color to grayscale, we do not just average red, green, and blue — human eyes are more sensitive to green than to blue, so we weight the channels:

$$\text{Gray} = 0.299R + 0.587G + 0.114B.$$

Here R, G, B are the red, green, and blue values of one pixel, each from 0 to 255.

✎ Worked example: converting one pixel to grayscale

Take a pixel with $R = 200$, $G = 120$, $B = 50$ (a warm, orange-ish color). What is its grayscale value?

Step 1. **Write down what we know.** $R = 200$, $G = 120$, $B = 50$.

Step 2. **Plug into the formula.**

$$\text{Gray} = 0.299(200) + 0.587(120) + 0.114(50).$$

Step 3. **Work out each piece.** $0.299 \times 200 = 59.8$. $0.587 \times 120 = 70.44$. $0.114 \times 50 = 5.7$.

Step 4. **Add them up.** $59.8 + 70.44 + 5.7 = 135.94$.

Step 5. **Round and sense-check.** Gray $\approx$ **136**, right in the middle of the 0–255 range — reasonable for a mid-brightness orange.

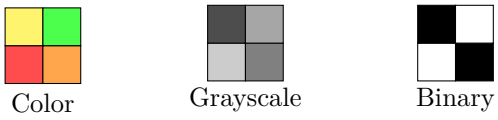


Figure 3: How to read it: the same 2×2 *patch, shown three ways. Color needs 3 numbers per cell, grayscale needs 1, binary needs only a single bit — — — black or white.*

Where this shows up in ML. Early digit-recognition models (like on the classic MNIST dataset) trained on grayscale images alone, since color carried no useful information for reading handwritten digits. Modern object detectors keep full color, because color itself is a clue — a red traffic light means something different from a green one.

✗ Watch out

A plain average, $(R + G + B)/3$, is not the same as the grayscale formula above. Plain averaging treats all three colors as equally bright to the eye, which they are not — green looks brighter to us than blue at the same numeric value.

✔ Key takeaway

Color, grayscale, and binary are the same grid idea at 3, 1, or 1-bit precision per pixel. Pick the smallest precision that still does the job.

4 Reading and reshaping images: libraries, resize, translate, rotate

Before a computer can "see" anything, code has to load the image, and often reshape it — resize it, move it, spin it, or write text on it. This is the toolbox layer.

☞ The intuition

OpenCV is the toolbox for pixel-level work: load, resize, move, draw. Deep-learning libraries like PyTorch, TensorFlow, and Keras sit one layer up: they take those pixels and train a model to recognize things in them.

★ Everyday picture

OpenCV is the workbench where you cut and prep the photo. PyTorch, TensorFlow, and Keras are the classroom where you teach a student (the model) to recognize what's in the prepped photo. You always prep before you teach.

Four everyday image installs and their commands:

Library	Install command
OpenCV	`pip install opencv-python`
PyTorch	`pip install torch torchvision`
TensorFlow	`pip install tensorflow`
Keras	`pip install keras`

Resizing scales every pixel position by a factor. Translation slides the whole image by a fixed amount, written as a small matrix:

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} + \begin{bmatrix} t_x \\ t_y \end{bmatrix},$$

where (x, y) is a pixel's original position, (x', y') is its new position, and (t_x, t_y) is how far we shift right and down.

✎ Worked example: shifting a pixel by translation

Shift an image by $t_x = 50$ pixels right and $t_y = 30$ pixels down. Where does the pixel originally at $(100, 100)$ end up?

Step 1. **Write down what we know.** $x = 100$, $y = 100$, $t_x = 50$, $t_y = 30$.

Step 2. **Apply the shift to x.** $x' = x + t_x = 100 + 50 = 150$.

Step 3. **Apply the shift to y.** $y' = y + t_y = 100 + 30 = 130$.

Step 4. **Sense-check.** The pixel moved right and down, exactly matching (t_x, t_y) being both positive — no rotation, no stretching, just a slide.

New position: **(150, 130)**.

✎ Worked example: resizing an image by a scale factor

An image is 400 pixels wide and 300 pixels tall. Resize it to half its size. What is the new size?

Step 1. **Write down what we know.** Width = 400, height = 300, scale = 0.5.

Step 2. **Scale the width.** $400 \times 0.5 = 200$.

Step 3. **Scale the height.** $300 \times 0.5 = 150$.

Step 4. **Sense-check.** Both dimensions shrank by the same factor, so the picture keeps its original proportions — nothing got stretched.

New size: **200 × 150**.

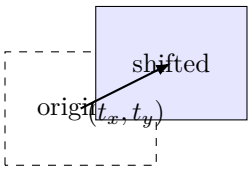


Figure 4: How to read it: translation slides every pixel by the same amount (t_x, t_y) — the whole rectangle moves as a rigid block, without changing shape.

Where this shows up in ML. Every CNN expects a fixed input size (like 224×224), so resizing is a mandatory preprocessing step before training or prediction. Random small rotations and translations are also used as *data augmentation*, teaching a model that a slightly shifted cat is still a cat.

✗ Watch out

Resizing without preserving the width-to-height ratio distorts objects — a circle can stretch into an oval. Translating a pixel off the edge of the image either crops it away or wraps it around, depending on the settings; forgetting which one your code uses can silently lose part of the picture.

✔ Key takeaway

Reading, resizing, moving, and rotating are pure number operations on the pixel grid. No "understanding" happens yet — that's still ahead.

5 Cleaning up noise: Gaussian and median filtering

Real photos have noise: random speckles from the camera sensor, bad lighting, or compression. Filters smooth this out before any serious analysis happens.

☞ The intuition

A Gaussian blur asks each pixel to blend with its neighbors, giving more weight to close neighbors and less to far ones. A median filter instead looks at a small neighborhood, sorts the values, and picks the middle one — ignoring extreme outliers completely.

★ Everyday picture

Gaussian blur is like averaging opinions in a room where people standing closer to you count more than people far away. A median filter is like a mediator polling a small group and reporting the "typical" answer — the one right in the middle — while ignoring the two wildly extreme opinions at either end.

A 3×3 Gaussian kernel puts the most weight on the center pixel and less on farther ones:

$$K = \frac{1}{16}\begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}.$$

To apply it, multiply each kernel weight by the pixel underneath it, add everything up, then divide by 16 (the sum of the weights) so brightness stays in the same range.

✎ Worked example: applying a Gaussian blur to one pixel

Take the 3×3 patch $\begin{bmatrix} 10 & 20 & 30 \\ 40 & 50 & 60 \\ 70 & 80 & 90 \end{bmatrix}$ and the Gaussian kernel above. What is the blurred value of the center pixel?

Step 1. **Write down what we know.** The patch values and the kernel weights $1, 2, 1, 2, 4, 2, 1, 2, 1$.

Step 2. **Multiply each patch value by its matching weight.** $1(10)+2(20)+1(30)+2(40)+4(50)+2(60)+1(70)+2(80)+1(90)$.

Step 3. **Work out each product.** $10, 40, 30, 80, 200, 120, 70, 160, 90$.

Step 4. **Add them all up.** $10+40+30+80+200+120+70+160+90 = 800$.

Step 5. **Divide by the weight total, 16.** $800/16 = 50$.

The blurred center value is **50** — exactly the original center value, because this patch was already smooth and symmetric. Blurring changes noisy patches far more than smooth ones.

✎ Worked example: removing salt-and-pepper noise with a median filter

Take the noisy 3×3 patch $\begin{bmatrix} 12 & 250 & 14 \\ 13 & 15 & 16 \\ 11 & 255 & 13 \end{bmatrix}$, where 250 and 255 are noise spikes. What does the median filter output for the center pixel?

Step 1. **Write down all 9 values.** $12, 250, 14, 13, 15, 16, 11, 255, 13$.

Step 2. **Sort them from smallest to largest.** $11, 12, 13, 13, 14, 15, 16, 250, 255$.

Step 3. **Find the middle value.** With 9 numbers, the middle one is the 5th: counting $11(1), 12(2), 13(3), 13(4), \mathbf{14}(5)$.

Step 4. **Sense-check.** The two noise spikes, 250 and 255, land at the very end of the sorted list and get ignored completely.

The median-filtered value is **14** — both noise spikes vanish, and the result stays close to the patch's real, quiet values.

Where this shows up in ML. Gaussian blurring is a standard data-augmentation and anti-aliasing step before shrinking images for a CNN, and it underlies the "Gaussian pyramids" used in classic

Gaussian kernel weights (before ÷16)

1	2	1
2	4	2
1	2	1

Figure 5: How to read it: the center weight (4) counts the most, the corners (1) count the least — a blur that trusts nearby pixels more than distant ones.

feature detectors like SIFT. Median filtering is still a common pre-cleaning step before feeding noisy medical or satellite images into a segmentation model.

✗ Watch out

A bigger blur kernel removes more noise, but it also blurs away real detail and edges — there's always a size trade-off. A median filter is excellent against extreme spikes but does little against mild, evenly spread noise.

✔ Key takeaway

Gaussian blur averages smoothly with more weight near the center. Median filtering picks the "middle vote" to kill outlier spikes. Match the filter to the noise.

6 Drawing the line: simple and adaptive thresholding

Thresholding turns a grayscale image into a binary one: every pixel becomes either fully black or fully white, based on one simple test.

☞ The intuition

Pick one cutoff brightness, T. Any pixel brighter than T becomes white (255); anything darker becomes black (0). That's simple thresholding — a single yes/no test for the whole image.

★ Everyday picture

Imagine sorting apples on a conveyor belt by ripeness, using one color cutoff. That works fine if the whole belt has the same lighting. But if half the belt sits in shadow, one fixed cutoff misjudges the shadowed apples. **Adaptive thresholding** is like giving each section of the belt its own judge, who picks a cutoff suited to that section's local lighting.

The rule for simple thresholding, for a pixel at position (x, y) with brightness $I(x, y)$:

$$\text{output}(x, y) = \begin{cases} 255 & \text{if } I(x, y) > T \\ 0 & \text{otherwise.} \end{cases}$$

Adaptive thresholding replaces the single, global T with a locally computed cutoff — for example, the average brightness of the small neighborhood around each pixel, minus a small constant.

✎ Worked example: simple thresholding on a 3×3 patch

Take the patch $\begin{bmatrix} 50 & 200 & 130 \\ 90 & 255 & 60 \\ 10 & 128 & 220 \end{bmatrix}$ and threshold $T = 127$. What is the binary output?

Step 1. **Write down the rule.** Above 127 becomes 255 (white); at or below 127 becomes 0 (black).

Step 2. **Check row 1.** $50 \le 127 \to 0$. $200 > 127 \to 255$. $130 > 127 \to 255$.

Step 3. **Check row 2.** $90 \le 127 \to 0$. $255 > 127 \to 255$. $60 \le 127 \to 0$.

Step 4. **Check row 3.** $10 \le 127 \to 0$. $128 > 127 \to 255$. $220 > 127 \to 255$.

Step 5. **Assemble the result.**

$$\begin{bmatrix} 0 & 255 & 255 \\ 0 & 255 & 0 \\ 0 & 255 & 255 \end{bmatrix}.$$

Every pixel is now purely black or white — the shading is gone, only the "brighter than 127" pattern survives.

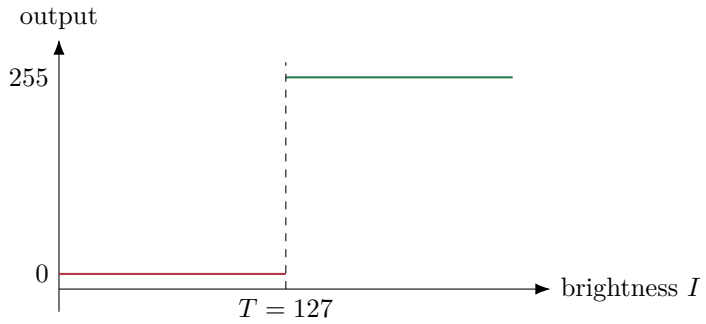


Figure 6: How to read it: below the threshold every pixel drops to 0 (black); at or above it, every pixel jumps to 255 (white) — one hard cutoff, no in-between.

Where this shows up in ML. Thresholding is still a fast, classic baseline for segmentation — separating dark text from a light page before OCR, for example. It is also used to turn a deep model's soft probability output into a hard yes/no mask, by thresholding the model's confidence score.

✗ Watch out

A single fixed threshold under uneven lighting can turn half an image solid black or solid white. If that happens, switch to adaptive thresholding instead of hunting for a "better" single number — there may not be one.

✔ Key takeaway

Thresholding is a one-number rule that turns shades of gray into a plain yes/no. Use a global cutoff under even lighting, an adaptive one otherwise.

7 Finding edges: convolution and kernels

An edge is a spot where brightness changes sharply — the outline of an object. We find edges by sliding a small filter, called a **kernel**, across the image and computing a weighted sum at every position. This sliding-and-summing operation is **convolution**.

☞ The intuition

A kernel is a tiny numeric "recipe" that reacts strongly to the exact pattern it's built to detect. An edge kernel stays near zero over flat, even regions, and spikes to a large number right where brightness jumps.

★ Everyday picture

Imagine a small stencil that compares the left side of a strip to the right side. If both sides are equally bright, the stencil reports zero — a flat area. If the left side is dark and the right side is bright, it reports a big number — an edge. That is exactly what an edge-detecting kernel does, pixel by pixel.

For a kernel K of size $r \times r$ and an image patch P of the same size, the convolution output is

$$\text{output} = \sum_{i,j} K(i,j)\, P(i,j),$$

one number per position of the kernel. If the input image is $n \times n$ and the kernel is $r \times r$, sliding the kernel everywhere (without padding) produces an output of size

$$(n - r + 1) \times (n - r + 1).$$

✎ Worked example: detecting an edge with a 3×3 kernel

This is the slide's own example. Take the horizontal-edge kernel $K = \begin{bmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{bmatrix}$ and the image patch $P = \begin{bmatrix} 10 & 50 & 90 \\ 20 & 60 & 100 \\ 30 & 70 & 110 \end{bmatrix}$. What is the convolution output?

Step 1. **Write down the formula.** Output $= \sum K(i,j)\, P(i,j)$ — multiply matching cells, then add.

Step 2. **Work out row 1.** $10(1) + 50(0) + 90(-1) = 10 - 90 = -80$.

Step 3. **Work out row 2.** $20(1) + 60(0) + 100(-1) = 20 - 100 = -80$.

Step 4. **Work out row 3.** $30(1) + 70(0) + 110(-1) = 30 - 110 = -80$.

Step 5. **Add the three rows.** $-80 + -80 + -80 = -240$.

The output is -240. A large negative number means brightness drops sharply from left to right across this patch — exactly the edge this kernel is built to find.

✎ Worked example: finding the output image's size

An image is 6×6 pixels ($n = 6$). We slide a 3×3 kernel ($r = 3$) across it with no padding. What size is the output?

Step 1. **Write down the formula.** Output size $= (n - r + 1) \times (n - r + 1)$.

Step 2. **Plug in the numbers.** $n = 6$, $r = 3$, so $n - r + 1 = 6 - 3 + 1 = 4$.

Step 3. **Sense-check.** A 3-wide kernel can only sit in 4 positions along a 6-wide row (starting at columns 1, 2, 3, or 4) before it runs off the edge.

The output is 4×4 — smaller than the input, because the kernel can't center itself right at the border.

Where this shows up in ML. This is exactly the first layer of a convolutional neural network. Instead of a hand-designed edge kernel, the network *learns* the numbers inside K from data, through backpropagation. The same size formula, extended with stride and padding, decides the output shape of every convolution layer in a CNN.

✗ Watch out

Near the border of the image, the kernel runs out of neighbors — this is exactly why the output shrinks, as the formula shows. Padding the image with a border of zeros first ("same" padding) keeps the output the same size as the input; forgetting whether your code pads or

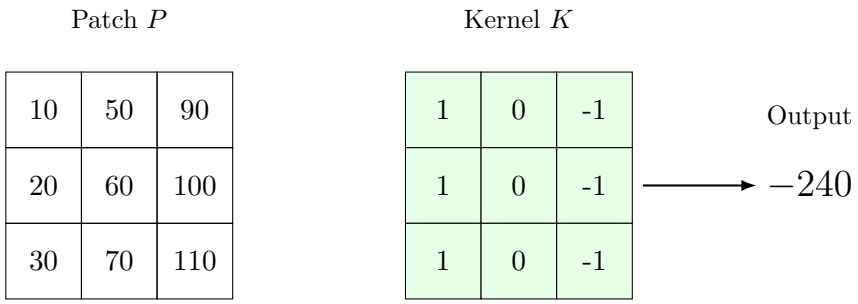


Figure 7: How to read it: multiply each cell of P by the matching cell of K and add everything up — a big negative number here means brightness drops sharply left to right: an edge.

not is a common source of size-mismatch errors.

✔ Key takeaway

A kernel is a tiny pattern-detector. Slide it everywhere — convolution — and its loud responses mark the edges. Without padding, the output always shrinks by $r - 1$ on each side.

8 A cleaner edge: the Canny recipe

A single kernel gives thick, noisy edge responses. **Canny edge detection** is a specific four-step recipe that cleans this up into crisp, one-pixel-wide outlines.

☞ The intuition

Canny is edge detection with a strict quality inspector at the end. It removes weak, blurry, and spurious responses, keeping only sharp, confident edges.

★ Everyday picture

A single kernel is like a rough charcoal sketch — thick, smudgy lines everywhere something changes. Canny is the fine pencil pass afterward: it thins those smudges down to one crisp line exactly where the real boundary is.

Canny runs in four steps, each cleaning up what came before:

Step	What it does
1	Gaussian blur the image first, to reduce noise before looking for edges.
2	Compute the brightness gradient (how fast, and in which direction, brightness changes).
3	Thin the edges down to one pixel wide (non-maximum suppression).
4	Keep only confident edges, and edges connected to them (hysteresis thresholding).

Step 4 uses *two* thresholds: a high one and a low one. A pixel is a real edge if its gradient is above the high threshold, *or* if it's above the low threshold and touches a pixel that is already confirmed as an edge.

✎ Worked example: hysteresis thresholding along one edge

Set low $= 50$, high $= 150$. Along a chain of five pixels, the gradient strengths are $180, 120, 40, 130, 20$. Which pixels survive as edges?

Step 1. **Pixel 1: strength 180.** $180 > 150 \rightarrow$ above the high threshold. Keep it as a confirmed edge.

Step 2. **Pixel 2: strength 120.** Between 50 and 150, so it needs a connection. It touches pixel 1, a confirmed edge. Keep it.

Step 3. **Pixel 3: strength 40.** Below 50, the low threshold. Discard it.

Step 4. **Pixel 4: strength 130.** Between 50 and 150 again, but it is *not* touching any confirmed edge (pixel 3 right next to it was just discarded). Discard it.

Step 5. **Pixel 5: strength 20.** Below 50. Discard it.

Only pixels **1 and 2** survive as edges — an isolated medium-strength pixel with no strong neighbor (pixel 4) gets thrown away, exactly as hysteresis intends.

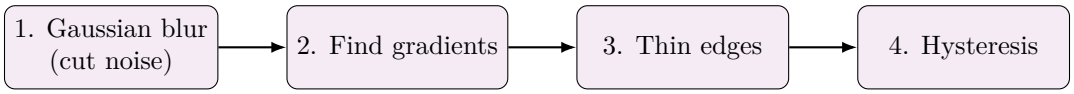


Figure 8: How to read it: Canny is four small, ordered steps, not one magic function — each step cleans up exactly what the previous step found.

Where this shows up in ML. Canny predates deep learning but is still used as a fast classical preprocessing step — for instance, adding an edge map as an extra input channel to a neural network, or in early lane-detection prototypes for self-driving cars. Modern deep networks mostly replace hand-crafted Canny with learned, edge-aware features instead.

✗ Watch out

Set the thresholds too high, and faint real edges get erased along with the noise. Set them too low, and background noise starts masquerading as edges. There is no universal setting — it depends on the image's contrast.

✔ Key takeaway

Canny is blur, then gradient, then thin, then connect only the confident edges. Two thresholds, working together, decide what survives.

9 Reshaping shapes: erosion and dilation

Once an image is binary (black and white only), **morphological operations** reshape the white regions using a small pattern called a **structuring element**.

☞ The intuition

Erosion shrinks white shapes: a pixel survives only if the whole structuring element fits inside the white region around it. Dilation grows white shapes: a pixel turns white if the structuring element touches *any* white pixel nearby.

★ Everyday picture

Erosion is like sandpaper wearing away the edges of a white shape — small, isolated noise specks get worn away completely. Dilation is like a shape growing a fuzzy border — tiny gaps and holes get filled in.

We use a *cross-shaped* structuring element here: a pixel plus its four direct neighbors (up, down, left, right) — five cells total, no corners.

✎ Worked example: eroding and dilating a noisy 5×5 patch

Take this 5×5 binary patch (1 = white, 0 = black), with a solid 3×3 white block and one

isolated noise pixel in the top-right corner:

$$\begin{bmatrix} 0 & 0 & 0 & 0 & 1 \\ 0 & 1 & 1 & 1 & 0 \\ 0 & 1 & 1 & 1 & 0 \\ 0 & 1 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

Erosion (a pixel stays 1 only if it, and all four cross-neighbors, are 1; anything off the grid counts as 0):

Step 1. **Test the noise pixel** (top-right corner). Its cross-neighbor above it falls outside the grid, which counts as 0. It fails the test. **It becomes 0** — the noise disappears.

Step 2. **Test the very center pixel of the block.** All four of its cross-neighbors are 1, and it is 1 itself. It passes. **It stays 1.**

Step 3. **Test a pixel on the block's edge**, say the top-middle of the block. Its cross-neighbor above it lies outside the block (value 0). It fails. **It becomes 0.**

Step 4. **Result.** Every border pixel of the block fails the same way, so only the single center pixel survives.

Erosion result: the whole block shrinks to **one surviving white pixel**, and the noise speck is gone entirely.

Dilation (a pixel becomes 1 if it, or any cross-neighbor, is 1):

Step 1. **Test the noise pixel's neighbors.** The pixel just below it and just to its left were both 0, but each now touches the noise pixel (which is 1). **Both turn 1** — the single speck grows into a small cluster.

Step 2. **Test a pixel just outside the block**, say directly above the block's top edge. It touches a 1 from the block below it. **It turns 1.**

Step 3. **Result.** The block gains a one-pixel-thick ring all around it, and the isolated noise speck grows slightly too.

Dilation grows every white region by one ring — including noise, which is why dilation alone is not a noise cleaner.

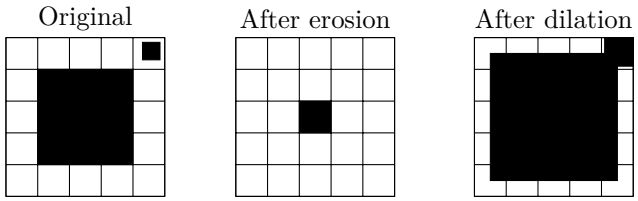


Figure 9: How to read it: the isolated noise speck (top right, original) vanishes completely after erosion, but grows into a small blob after dilation — the main block only shrinks (erosion) or grows (dilation) at its border.

Where this shows up in ML. Morphological cleanup is a standard post-processing step after a deep segmentation model produces a binary mask — eroding away speckle noise, then dilating back (an operation called *opening*) to restore the true boundary without the noise.

✗ Watch out

Erode too aggressively (too many times, or too big a structuring element) and small *real* objects disappear along with the noise. Dilate too aggressively and two separate nearby objects can merge into one blob.

✔ Key takeaway

Erosion shrinks and cleans; dilation grows and fills. Erosion followed by dilation — *opening* — removes small noise without losing the real shape.

Quick self-test

Five questions straight from the lecture's own quiz, with the section that explains each answer.

Question	Answer	See
700 wide, 550 tall, 3 channels — NumPy shape?	$(550, 700, 3)$	§2
Total pixels in a 393×312 image?	122,616	§2
OpenCV stores color pixels in what order?	BGR	§2
Main goal of computer vision?	Understand and interpret visual information	§1
Purpose of edge detection?	Identify boundaries within an image	§7

Glossary & symbols

Key terms.

pixel one cell of an image's number grid; its value is how bright that tiny spot is.

channel one of the color layers (red, green, blue) stacked to make a color image.

grayscale an image with one brightness number per pixel, no color.

binary image an image where every pixel is only fully black or fully white.

kernel a small grid of numbers slid across an image to detect a pattern, like an edge.

convolution multiplying a kernel by the image underneath it and adding the results, at every position.

threshold a single brightness cutoff used to split an image into black and white.

hysteresis Canny's rule for keeping edge pixels connected to a strong edge, even if they are individually weaker.

structuring element the small shape used by erosion and dilation to test each pixel's neighborhood.

erosion shrinking white regions in a binary image; kills small isolated noise.

dilation growing white regions in a binary image; fills small gaps.

Symbols at a glance.

Symbol	Meaning
H, W, C	height, width, and channel count of an image array
$I(x, y)$	the brightness value of the pixel at position (x, y)
T	the threshold: the cutoff brightness used to make a pixel black or white
K	a convolution kernel: a small grid of numbers that detects a pattern
n, r	the input image size ($n \times n$) and the kernel size ($r \times r$)
t_x, t_y	how far an image is shifted right and down during translation

Further reading

- Adrian Kaehler and Gary Bradski, *Learning OpenCV 4: Computer Vision with Python 3*.
- Joseph Howse, *OpenCV 4 for Secret Agents*.
- Joseph Howse and Prateek Joshi, *OpenCV 4 Computer Vision Projects with Python 3*.
- Oscar Deniz Suarez, María del Milagro Fernandez Carrobles, and Noelia Vallez Enano, *OpenCV Essentials*.